СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1. ОБЩАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ 4

1.1. Организационная структура предприятия 4

1.2. Внутренний распорядок работы предприятия, охрана труда ИТ-специалистов 5

1.3. Должностные инструкции ИТ-специалистов предприятия 5

2. РЕВЬЮИРОВАНИЕ ПРОГРАММНЫХ ПРОДУКТОВ 6

2.1. Ревьюирование программного кода в соответствии с технической документацией 6

2.2. Измерение характеристик компонент программного продукта 6

2.3. Исследование созданного программного кода с использованием специализированных программных средств 7

2.4. Сравнительный анализ программных продуктов и средств разработки 7

3. ВЫПОЛНЯЕМЫЕ ЗАДАНИЯ 8

3.1. Краткое описание разработанного программного продукта 8

3.2. Анализ средств разработки 9

3.3. Разработка структуры и архитектуры приложения 9

3.4. Разработка пользовательского интерфейса и загрузка изображений 10

3.5. Реализация функций обработки изображений 11

3.6. Реализация сохранения результата и дополнительных функций 11

3.7. Автоматизированное тестирование 12

3.8. Измерение характеристик программного продукта 12

3.8.1 Сравнение размеров файлов 13

3.9. Подготовка документации проекта 14

ЗАКЛЮЧЕНИЕ 15

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 16

ПРИЛОЖЕНИЕ А 17

ВВЕДЕНИЕ

Прохождение производственной практики в ООО «Малленом Системс» стало для меня возможностью применить полученные в техникуме знания на реальном проекте. В рамках задания мне предстояло разработать настольное приложение для обработки изображений с графическим интерфейсом, что потребовало не только навыков программирования, но и понимания принципов проектирования архитектуры и тестирования.

Актуальность выбранной темы обусловлена тем, что работа с графикой является одной из самых востребованных задач в современных десктопных приложениях, а грамотное разделение интерфейса и бизнес-логики — залог того, что программу будет легко поддерживать и развивать в будущем.

Целью практики является разработка программного продукта на языке Python с использованием фреймворка PyQt5 и библиотеки Pillow, а также проведение анализа его характеристик.

Для достижения поставленной цели были решены следующие задачи:

- провести обратное проектирование (построены UML-диаграммы: компонентов, деятельности, последовательности и прецедентов);
- реализовать пользовательский интерфейс и визуальным выделением области обрезки;
- написать модуль обработки изображений (ImageProcessor) и модуль логирования/истории (AppStateManager);
- провести замеры скоростных показателей операций с использованием стандартной библиотеки time;

1. ОБЩАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ

ООО «Малленом Системс» специализируется на разработке программного обеспечения и внедрении комплексных технических решений. Компания выполняет задачи по автоматизации производственных процессов, занимается машинным зрением и разработкой собственных программных продуктов.

1.1. Организационная структура предприятия

Организационная структура компании является линейной. Ключевым подразделением для моего проекта является Центр по развитию интеллектуальных систем (отдел разработки ПО), где создаются и оптимизируются программные продукты. Также важную роль играет отдел технической поддержки и контроля качества, который занимается тестированием ПО и выявлением ошибок, с чем мне пришлось тесно столкнуться при написании модульных тестов для своего приложения.

Кроме того, в структуру компании входят:

- Производственно-технический отдел - выполнение пусконаладочных работ на предприятиях, проектирование размещения оборудования и его установка.
- АУП (административно-управленческий персонал) - общее управление компанией, планирование деятельности, формирование стратегии развития и организация внешних коммуникаций.
- Группа маркетинга - разработка маркетинговой стратегии, продвижение бренда и продуктов.
- Коммерческий отдел - взаимодействие с заказчиками, продажа продуктов и поиск новых клиентов.

- Отдел кадров - подбор, адаптация и сопровождение сотрудников, ведение кадрового документооборота.
- Юридический отдел - работа с документами и договорными обязательствами в соответствии с законодательством.
- Бухгалтерия - ведение бухгалтерского учёта и финансово-экономической деятельности.
- ОХР (общественно-хозяйственные рабочие) - поддержание порядка, уборка и обслуживание служебных помещений.

1.2. Внутренний распорядок работы предприятия, охрана труда ИТ-специалистов

В компании действует стандартный пятидневный график с 9:00 до 18:00. Особое внимание уделяется охране труда. Перед началом практики я прошел вводный инструктаж. Согласно результатам спец оценки, рабочие места в ИТ-отделе не имеют вредных производственных факторов. Тем не менее, при работе за компьютером необходимо соблюдать режим труда и отдыха, а также следить за эргономикой рабочего места для предотвращения зрительного и мышечного утомления.

1.3. Должностные инструкции ИТ-специалистов предприятия

В процессе практики я ознакомился с тем, как организована работа ИТ-специалистов. Разработчики занимаются написанием кода и исправлением ошибок. Специалисты по контролю качества проверяют код, проводят тестирование и описывают найденные дефекты. Например, при ревьюировании моего кода и модульных тестов оценивались не только работоспособность, но и читаемость кода.

2. РЕВЬЮИРОВАНИЕ ПРОГРАММНЫХ ПРОДУКТОВ

Ревьюирование кода — это процесс проверки программного обеспечения на соответствие требованиям технического задания, а также поиск потенциальных ошибок и уязвимостей.

2.1. Ревьюирование программного кода в соответствии с технической документацией

Мой проект разрабатывался строго в соответствии с техническим заданием (ТЗ). В ходе ревью было проверено, что все заявленные функции реализованы. Например, в ТЗ требовалось обрабатывать ошибки ввода-вывода (FileNotFoundError, IOError) с выводом графического сообщения. В моем коде это реализовано через блоки try-except в классе MainWindow с последующим вызовом QMessageBox.critical(). Кроме того, история действий ведется строго в формате JSON, как и требовалось.

2.2. Измерение характеристик компонент программного продукта

Для оценки объема работ использовались метрики количества строк кода. Главный модуль проекта (project_3.py) занимает около 300 строк, включая интерфейс и логику. Модуль тестирования (test_image_processor.py) — около 70 строк. Основная логика инкапсулирована в классе ImageProcessor, который не зависит от интерфейса, что позволяет легко переиспользовать его в других проектах или тестировать отдельно.

2.3. Исследование созданного программного кода с использованием специализированных программных средств

В качестве основного инструмента для анализа кода я использовал встроенный в VS Code анализатор Pylance. Он помог выявить неиспользуемые импорты и несоответствия типов данных. Это позволило сделать код более аккуратным и соответствующим общепринятым стандартам Python.

2.4. Сравнительный анализ программных продуктов и средств разработки

Изначально рассматривался вариант реализации на C# с использованием WPF, однако выбор был остановлен на связке Python + PyQt5. Это обусловлено тем, что Python позволяет быстрее писать и отлаживать код, а библиотека Pillow предоставляет очень удобный и лаконичный интерфейс для работы с изображениями (например, методы crop и resize), что в итоге сократило объем кода и время разработки.

3. ВЫПОЛНЯЕМЫЕ ЗАДАНИЯ

3.1. Краткое описание разработанного программного продукта

Разработанное приложение представляет собой десктопную утилиту для обработки изображений. Программа позволяет загружать изображения (кнопкой или перетаскиванием), просматривать их, получать разрешение и формат файла. Реализованы такие операции, как обрезка выделенной области, перевод в черно-белый формат, изменение разрешения, отмена последнего действия и сохранение результата в папку output.

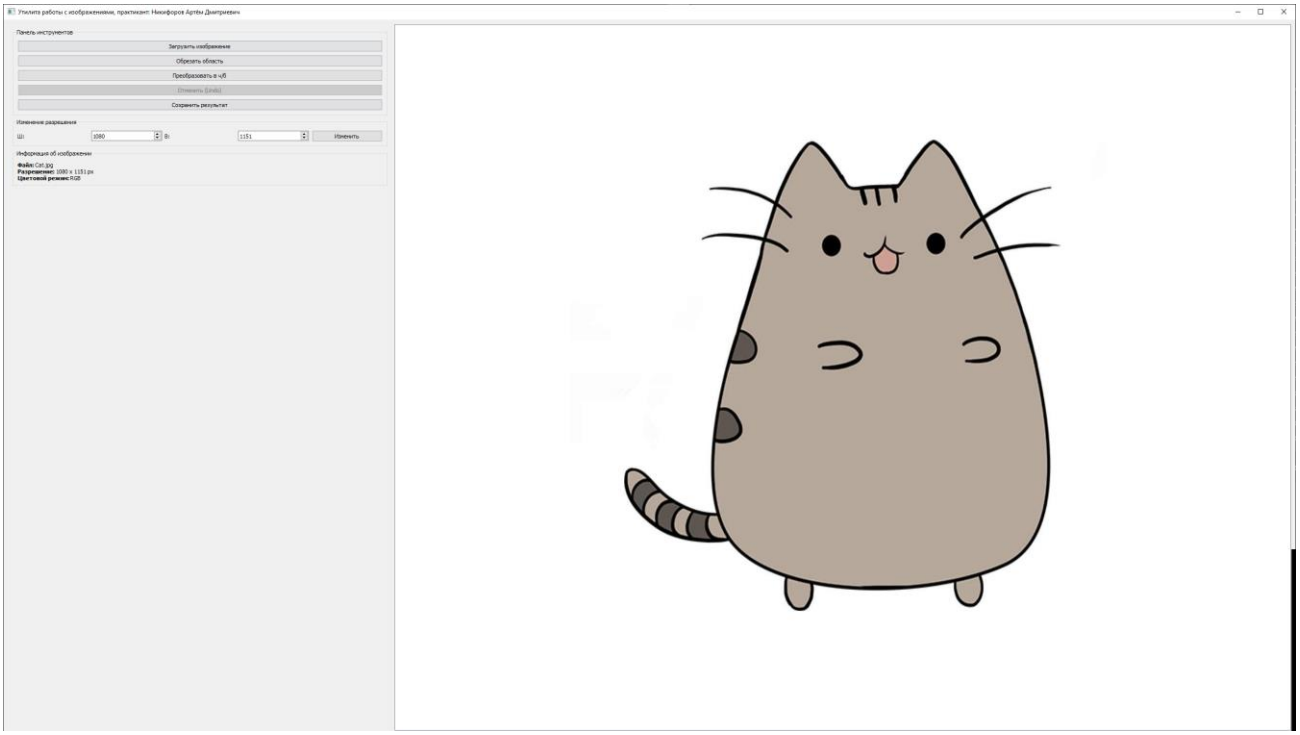


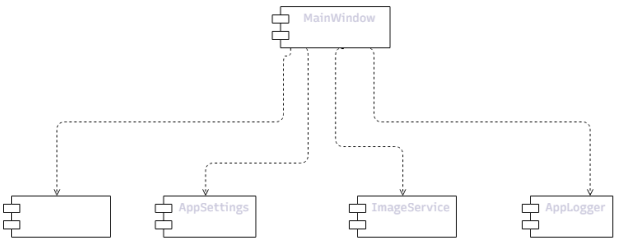
Рисунок 1 – Главное окно приложения project_3.py

3.2. Анализ средств разработки

Для создания интерфейса использовался PyQt5. Этот фреймворк позволяет создавать профессиональные интерфейсы, поддерживает работу с событиями мыши (что критично для обрезки). Обработка изображений базируется на библиотеке Pillow (PIL), которая отлично справляется с чтением, изменением размера (использовался качественный фильтр LANCZOS) и сохранением файлов. В качестве среды разработки использовался VS Code с расширениями для Python.

3.3. Разработка структуры и архитектуры приложения

Приложение спроектировано по принципу разделения ответственности (Separation of Concerns). Интерфейс (MainWindow) отделен от логики обработки (ImageProcessor). Это позволяет менять интерфейс, не затрагивая логику работы с картинками. Также были выделены вспомогательные модули: AppStateManager (работа с JSON для истории и настроек) и AppLogger (логирование событий в текстовый файл).



Модуль	Описание
MainWindow	Модуль пользовательского интерфейса. Обеспечивает загрузку и отображение изображений, выбор фильтров, обрезку, изменение размеров, сохранение результата и взаимодействие с другими модулями.
ImageService	Модуль обработки изображений. Выполняет обрезку, перевод в оттенки серого (Grayscale), изменение размеров и получение характеристик изображения.
AppSettings	Модуль сохранения и восстановления настроек приложения. Хранит путь к последнему изображению, параметры фильтров, размеры изображения и размеры окна программы.
ActionHistory	Модуль ведения истории действий. Записывает даты, названия операций и их параметры в JSON-файл.
AppLogger	Модуль журналирования. Записывает события работы приложения и сообщения об ошибках в файл журнала.

Рисунок 2 - UML-диаграмма структуры модулей приложения project_3.py

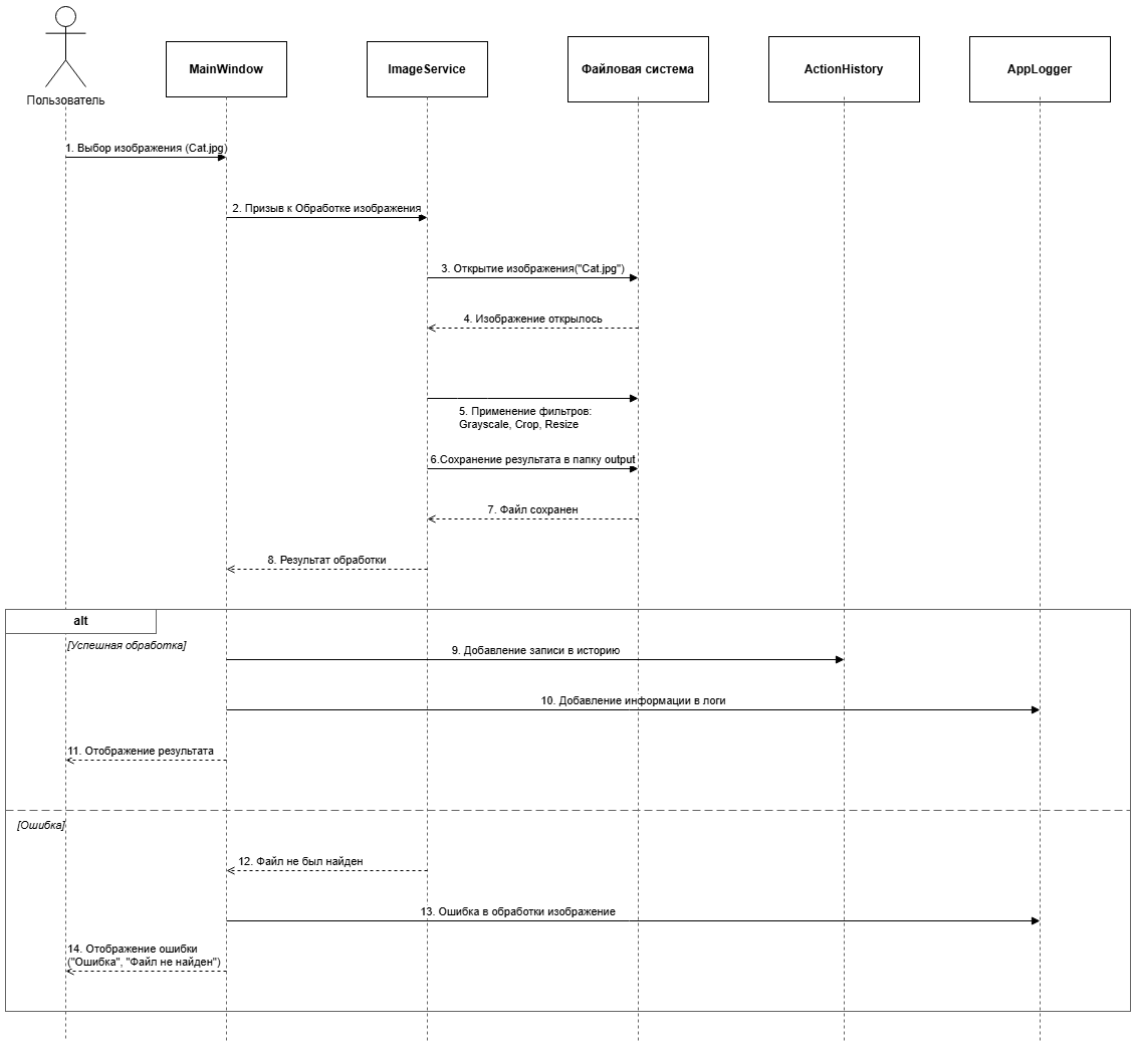


Рисунок 3 - Диаграмма последовательности обработки изображения

3.4. Разработка пользовательского интерфейса и загрузка изображений

Главное окно разделено на две части: слева панель инструментов, справа рабочая область. Рабочая область реализована на базе QGraphicsView, что позволяет удобно масштабировать изображение. Для загрузки файлов используется QFileDialog. Также реализован Drag & Drop: при перетаскивании картинки в окно срабатывает событие dropEvent, которое считывает путь и загружает файл. Для выделения области обрезки используется виджет QRubberBand, который рисует пунктирную рамку поверх изображения.

3.5. Реализация функций обработки изображений

Класс ImageProcessor содержит статические методы для каждой операции. Обрезка выполняется через img.crop(box), где box — координаты, полученные из QRubberBand (пересчитанные из координат виджета в координаты самого изображения). Перевод в ч/б реализован через ImageOps.grayscale(). Изменение размера выполняется методом resize() с указанием новых размеров и фильтра LANCZOS для сглаживания.

3.6. Реализация сохранения результата и дополнительных функций

Сохранение происходит через QFileDialog.getSaveFileName. По умолчанию предлагается папка output. Каждая операция (загрузка, обрезка, фильтр, сохранение) автоматически записывается в файл configs/history.json с указанием даты и параметров. Это позволяет в любой момент посмотреть историю действий. Функция «Отмена» (Undo) сохраняет копию предыдущего изображения в оперативную память и возвращает её при нажатии.

3.7. Автоматизированное тестирование

Для проверки корректности работы логики был создан файл test_image_processor.py с использованием стандартной библиотеки unittest. Написано 5 тестов: проверка чтения метаданных, обработка ошибки отсутствия файла, конвертация в ч/б, обрезка и изменение размера. Тесты запускаются из терминала и позволяют убедиться, что после внесения изменений в код ничего не сломалось.

3.8. Измерение характеристик программного продукта

Для замеров использовался скрипт benchmark.py на базе модуля time (метод perf_counter). Замеры проводились на изображении Cat.jpg (1080x1151). Результаты представлены в таблице 1.

Таблица 1 — Сравнение общего времени выполнения операций

Операции/количество тестов	1	2	3	4	5	Среднее
Загрузка	13.92	13.32	13.64	13.60	13.63	13.6 мс
Ч/Б (Grayscale)	1.20	1,44	1,24	1,41	1,35	1.3 мс
Изменение размера	17.20	16.46	16.33	16.42	16.22	16.5 мс
Обрезка (Crop)	0.33	0.33	0.33	0.42	0.34	0.3 мс
Сохранение	13.96	17.13	6,1	6,53	6,79	10.1 мс
Общее время всех операций	46.61	48.68	37.64	38.39	38.33	42.0 мс

Как видно из таблицы, среднее время полного цикла обработки составило 42.0 мс. Самыми быстрыми оказались операции с самим изображением (Crop и Grayscale), а основное время уходит на дисковый ввод-вывод (загрузка и сохранение). Это говорит о высокой оптимизации кода обработки.

3.8.1 Сравнение размеров файлов

Был проведен анализ размеров исходного кода и служебных файлов (таблица 2).

Таблица 2 — Сравнение размеров файлов проекта

№	Файл	Размер
1	project_3.py	16 КБ
2	test_image_processor.py	3 КБ
3	app.log	1 КБ
4	history.json	2 КБ
5	settings.json	1 КБ
6	benchmark.py	4 КБ

3.9. Подготовка документации проекта

Для удобства работы с проектом был создан файл README.md. В нем описаны требования к окружению, инструкция по установке зависимостей и команды для запуска приложения и тестов. Весь код снабжен комментариями, объясняющими назначение функций.

Дополнительно был подготовлен документ «Архитектурные решения и технологии», который описывал архитектуру приложения, назначение его основных компонентов и применяемые технологии.

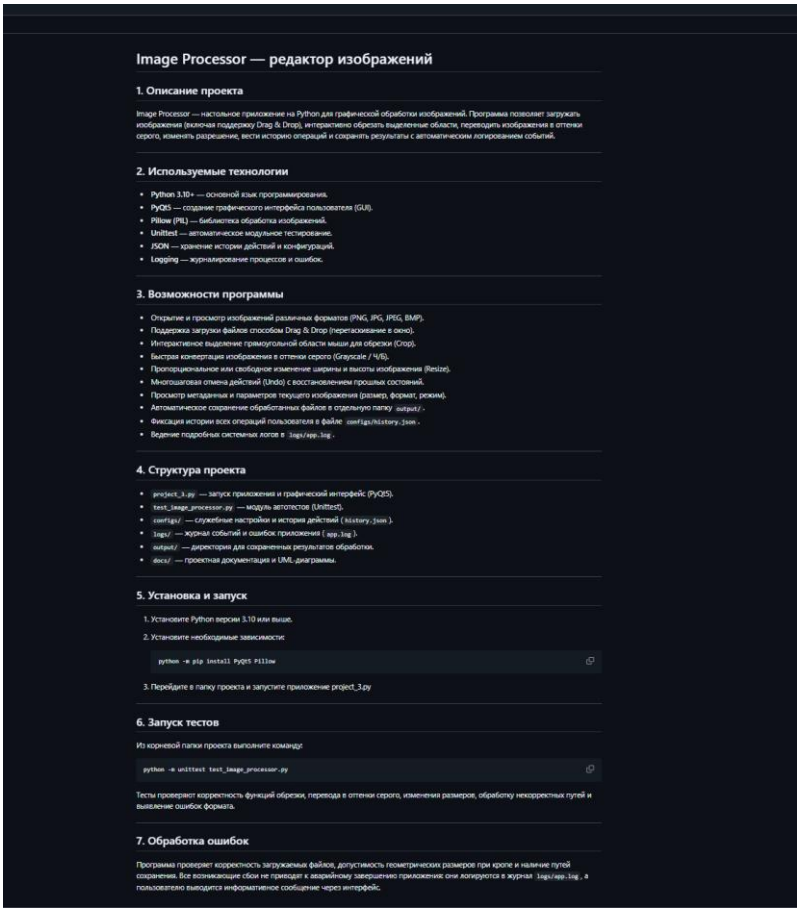


Image Processor — редактор изображений

1. Описание проекта

Image Processor — настольное приложение на Python для графической обработки изображений. Программа позволяет загружать изображения (включая поддержку Drag & Drop), интерактивно обрезать выделенные области, переводить изображения в оттенки серого, изменять разрешение, вести историю операций и сохранять результаты с автоматическим логированием событий.

2. Используемые технологии

- **Python 3.10+** — основной язык программирования.
- **PyQt5** — создание графического интерфейса пользователя (GUI).
- **Pillow (PIL)** — библиотека обработка изображений.
- **Unittest** — автоматическое модульное тестирование.
- **JSON** — хранение истории действий и конфигураций.
- **Logging** — журналирование процессов и ошибок.

3. Возможности программы

- Открытие и просмотр изображений различных форматов (PNG, JPG, JPEG, BMP).
- Поддержка загрузки файлов способом Drag & Drop (перетаскивание в окно).
- Интерактивное выделение прямоугольной области мыши для обрезки (Crop).
- Быстрая конвертация изображения в оттенки серого (Grayscale / Ч/Б).
- Пропорциональное или свободное изменение ширины и высоты изображения (Resize).
- Многошаговая отмена действий (Undo) с восстановлением прошлых состояний.
- Просмотр метаданных и параметров текущего изображения (размер, формат, режим).
- Автоматическое сохранение обработанных файлов в отдельную папку `output/`.
- Фиксация истории всех операций пользователя в файле `configs/history.json`.
- Ведение подробных системных логов в `logs/app.log`.

4. Структура проекта

- `project_3.py` — запуск приложения и графический интерфейс (PyQt5).
- `test_image_processor.py` — модуль автотестов (Unittest).
- `configs/` — служебные настройки и история действий (`history.json`).
- `logs/` — журнал событий и ошибок приложения (`app.log`).
- `output/` — директория для сохраненных результатов обработок.
- `docs/` — проектная документация и UML диаграммы.

5. Установка и запуск

1. Установите Python версии 3.10 или выше.
2. Установите необходимые зависимости:

```
python -m pip install PyQt5 Pillow
```

3. Перейдите в папку проекта и запустите приложение project_3.py

6. Запуск тестов

Из корневой папки проекта выполните команду:

```
python -m unittest test_image_processor.py
```

Тесты проверяют корректность функций обрезки, перевода в оттенки серого, изменения размеров, обработку некорректных путей и выявление ошибок формата.

7. Обработка ошибок

Программа проверяет корректность загружаемых файлов, допустимость геометрических размеров при кропе и наличие путей сохранения. Все возникающие сбои не приводят к аварийному завершению приложения: они логируются в журнал `logs/app.log`, а пользователю выводится информативное сообщение через интерфейс.

Рисунок 4 - Документация проекта в файле README.md

ЗАКЛЮЧЕНИЕ

В ходе прохождения производственной практики была выполнена работа по разработке настольного программного продукта (утилиты для обработки изображений), предназначенного для загрузки, редактирования и сохранения графических файлов. В процессе выполнения заданий были изучены и применены средства разработки на языке Python с использованием фреймворка PyQt5 и библиотеки Pillow.

В рамках практики была создана модульная структура приложения, включающая главный модуль project_3.py, модуль обработки изображений (ImageProcessor), модуль управления историей и настройками (AppStateManager), а также скрипт автоматизированного тестирования test_image_processor.py. Такое разделение позволило разграничить пользовательский интерфейс, логику обработки изображений и тестирование. Были реализованы основные функции приложения: загрузка изображений (в том числе методом, отображение их характеристик, преобразование в черно-белый формат (Grayscale), обрезка выделенной области (Crop), изменение разрешения (Resize), отмена действий (Undo) и сохранение обработанного результата.

В результате выполненной работы были закреплены практические навыки разработки приложений на Python, организации структуры программного решения, создания графического интерфейса, использования сторонних библиотек, подготовки технической документации и оценки характеристик программного продукта.

Таким образом, цели производственной практики были реализованы посредством разработки приложения для обработки изображений и выполнения задач.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Руководство по Python [Электронный ресурс]. — URL: https://docs.python.org/ru/3/tutorial/index.html (дата обращения: 25.10.2026).
2. PyQt5 — Документация для разработчиков [Электронный ресурс]. — URL: https://docs.astralinux.ru/latest/desktop/python/pyqt5/ (дата обращения: 25.10.2026).
3. Документация по Pillow [Электронный ресурс]. — URL: https://pillow.readthedocs.io/en/stable/ (дата обращения: 25.10.2026).
4. UML-диаграммы: виды и примеры [Электронный ресурс]. — URL: https://practicum.yandex.ru/blog/uml-diagrammy/ (дата обращения: 25.10.2026).
5. Документация о проверке выполнения скрипта на Python [Электронный ресурс]. — URL: https://www.geeksforgeeks.org/python/how-to-check-the-execution-time-of-python-script/ (дата обращения: 25.10.2026).
6. Документация о unittest [Электронный ресурс]. — URL: https://docs.python.org/3/library/unittest.html (дата обращения: 29.10.2026).
7. Документация о инструментов работы с изображениями в Python [Электронный ресурс]. — URL: https://blog.skillfactory.ru/rabota-s-izobrazheniyami-v-python-i-pillow/ (дата обращения: 25.10.2026).
8. Пакет logging, ведение журнала в Python [Электронный ресурс]. — URL: https://docs-python.ru/standart-library/paket-logging-python/ (дата обращения: 25.10.2026).
9. Модуль json — работа с форматом JSON в Python [Электронный ресурс]. — URL: https://docs.python.org/3/library/json.html (дата обращения: 25.10.2026).
10. Работа с файлами в Python: чтение и запись [Электронный ресурс]. — URL: https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files (дата обращения: 25.10.2026).

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

ФАЙЛ 1. Project_3.py:

```
import sys

import os

import json

import logging

from datetime import datetime

from PIL import Image, UnidentifiedImageError, ImageOps


from PyQt5.QtWidgets import (

    QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,

    QPushButton, QLabel, QFileDialog, QSpinBox, QMessageBox,

    QGraphicsView, QGraphicsScene, QRubberBand,

    QFrame, QGroupBox, QSplitter

)

from PyQt5.QtGui import QPixmap, QImage

from PyQt5.QtCore import Qt, QRect, QPoint, QSize


def ensure_directories_exist():
```

```
        """Автоматическое создание всех необходимых служебных директорий."""
        base_dir = os.path.dirname(os.path.abspath(__file__))
        for folder in ["logs", "configs", "output"]:
            dir_path = os.path.join(base_dir, folder)
            os.makedirs(dir_path, exist_ok=True)

    ensure_directories_exist()

    # Настройка логирования
    logging.basicConfig(
        filename=os.path.join(os.path.dirname(os.path.abspath(__file__)), "logs", "app.log"),
        level=logging.INFO,
        format="%(asctime)s - %(levelname)s - %(message)s"
    )

    class ImageProcessor:
        """Библиотека обработки изображений."""
```

```
    @staticmethod
    def get_info(file_path: str) -> dict:
        """Получение информации о файле."""
        if not os.path.exists(file_path):
            raise FileNotFoundError(f"Файл не найден: {file_path}")

        try:
            with Image.open(file_path) as img:
                return {
                    "width": img.width,
                    "height": img.height,
                    "format": img.format if img.format else "UNKNOWN",
                    "size_bytes": os.path.getsize(file_path)
                }
        except UnidentifiedImageError:
            raise ValueError("Выбранный файл не является поддерживаемым изображением.")

    @staticmethod
    def convert_to_grayscale(img: Image.Image) -> Image.Image:
```

```
        """Преобразование в черно-белый формат (Grayscale)."""
        logging.info("Применение фильтра: Перевод в оттенки серого (ч/б)")
        return ImageOps.grayscale(img).convert("RGB")

    @staticmethod
    def crop(img: Image.Image, box: tuple) -> Image.Image:
        """Обрезка (crop) прямоугольной области: box = (left, upper, right, lower)."""
        logging.info(f"Выполнение обрезки области: {box}")
        return img.crop(box)

    @staticmethod
    def resize(img: Image.Image, width: int, height: int) -> Image.Image:
        """Изменение разрешения / размера изображения."""
        logging.info(f"Изменение разрешения до {width}x{height}")
        return img.resize((width, height), Image.Resampling.LANCZOS)

class AppStateManager:
    """Управление файлами конфигурации и историей действий пользователя."""
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
CONFIG_PATH = os.path.join(BASE_DIR, "configs", "settings.json")
HISTORY_PATH = os.path.join(BASE_DIR, "configs", "history.json")

@classmethod
def load_settings(cls) -> dict:
    if os.path.exists(cls.CONFIG_PATH):
        try:
            with open(cls.CONFIG_PATH, "r", encoding="utf-8") as f:
                return json.load(f)
        except Exception:
            pass
    return {"last_path": "", "window_size": [1000, 650]}

@classmethod
def save_settings(cls, settings: dict):
    with open(cls.CONFIG_PATH, "w", encoding="utf-8") as f:
        json.dump(settings, f, ensure_ascii=False, indent=4)

@classmethod
```

```
    def append_history(cls, operation: str, details: str):
        history = []
        if os.path.exists(cls.HISTORY_PATH):
            try:
                with open(cls.HISTORY_PATH, "r", encoding="utf-8") as f:
                    history = json.load(f)
            except Exception:
                history = []

        record = {
            "datetime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "operation": operation,
            "details": details
        }
        history.append(record)

        with open(cls.HISTORY_PATH, "w", encoding="utf-8") as f:
            json.dump(history, f, ensure_ascii=False, indent=4)


class ImageAreaWidget(QGraphicsView):
```

```
    def __init__(self, parent=None):
        super().__init__(parent)
        self.scene = QGraphicsScene(self)
        self.setScene(self.scene)
        self.pixmap_item = None
        self.setAcceptDrops(True)

        self.rubberBand = None
        self.origin = QPoint()
        self.crop_rect_pixels = None

    def set_image(self, pil_img: Image.Image):
        self.scene.clear()
        self.crop_rect_pixels = None

        if pil_img.mode != "RGB":
            pil_img = pil_img.convert("RGB")
        data = pil_img.tobytes("raw", "RGB")
        qimg = QImage(data, pil_img.width, pil_img.height, pil_img.width * 3,
QImage.Format_RGB888)
```

```
            pixmap = QPixmap.fromImage(qimg)

            self.pixmap_item = self.scene.addPixmap(pixmap)
            self.setSceneRect(0, 0, pil_img.width, pil_img.height)

        def mousePressEvent(self, event):
            if event.button() == Qt.LeftButton and self.pixmap_item:
                self.origin = event.pos()
                if not self.rubberBand:
                    self.rubberBand = QRubberBand(QRubberBand.Rectangle, self)
                self.rubberBand.setGeometry(QRect(self.origin, QSize()))
                self.rubberBand.show()
            super().mousePressEvent(event)

        def mouseMoveEvent(self, event):
            if self.rubberBand and self.rubberBand.isVisible():
                                    self.rubberBand.setGeometry(QRect(self.origin,
event.pos()).normalized())
            super().mouseMoveEvent(event)
```

```
def mouseReleaseEvent(self, event):
    if self.rubberBand and self.rubberBand.isVisible() and self.pixmap_item:
        rect = self.rubberBand.geometry()
        scene_rect = self.mapToScene(rect).boundingRect()

        left = max(0, int(scene_rect.left()))
        top = max(0, int(scene_rect.top()))
        right = min(int(self.scene.width()), int(scene_rect.right()))
        bottom = min(int(self.scene.height()), int(scene_rect.bottom()))

        if right > left and bottom > top:
            self.crop_rect_pixels = (left, top, right, bottom)
        else:
            self.crop_rect_pixels = None
    super().mouseReleaseEvent(event)

def dragEnterEvent(self, event):
    if event.mimeData().hasUrls():
        event.acceptProposedAction()
```

```
        def dropEvent(self, event):
            for url in event.mimeData().urls():
                file_path = url.toLocalFile()
                if os.path.isfile(file_path):
                    self.window().load_image_from_path(file_path)
                    break


    class MainWindow(QMainWindow):
        def __init__(self):
            super().__init__()
             self.setWindowTitle("Утилита работы с изображениями, практикант: Никифоров Артём Дмитриевич")

            self.settings = AppStateManager.load_settings()
            self.resize(self.settings.get("window_size", [1000, 650])[0],
                        self.settings.get("window_size", [1000, 650])[1])

            self.current_pil_image = None
            self.previous_pil_image = None
            self.current_file_path = ""
```

```
        self.init_ui()


    def init_ui(self):
        main_widget = QWidget()
        self.setCentralWidget(main_widget)
        main_layout = QHBoxLayout(main_widget)

        splitter = QSplitter(Qt.Horizontal)
        main_layout.addWidget(splitter)

        left_panel = QFrame()
        left_layout = QVBoxLayout(left_panel)

        # Панель инструментов
        btn_box = QGroupBox("Панель инструментов")
        btn_layout = QVBoxLayout(btn_box)

        self.btn_load = QPushButton("Загрузить изображение")
        self.btn_load.clicked.connect(self.on_load_clicked)
```

```
self.btn_crop = QPushButton("Обрезать область")
self.btn_crop.clicked.connect(self.on_crop_clicked)

self.btn_grayscale = QPushButton("Преобразовать в ч/б")
self.btn_grayscale.clicked.connect(self.on_grayscale_clicked)

self.btn_undo = QPushButton("Отменить (Undo)")
self.btn_undo.setEnabled(False)
self.btn_undo.clicked.connect(self.on_undo_clicked)

self.btn_save = QPushButton("Сохранить результат")
self.btn_save.clicked.connect(self.on_save_clicked)

btn_layout.addWidget(self.btn_load)
btn_layout.addWidget(self.btn_crop)
btn_layout.addWidget(self.btn_grayscale)
btn_layout.addWidget(self.btn_undo)
btn_layout.addWidget(self.btn_save)
```

```
# Изменение размера
resize_box = QGroupBox("Изменение разрешения")
resize_layout = QHBoxLayout(resize_box)
self.spin_w = QSpinBox()
self.spin_w.setRange(1, 10000)
self.spin_h = QSpinBox()
self.spin_h.setRange(1, 10000)
btn_resize = QPushButton("Изменить")
btn_resize.clicked.connect(self.on_resize_clicked)

resize_layout.addWidget(QLabel("Ш:"))
resize_layout.addWidget(self.spin_w)
resize_layout.addWidget(QLabel("В:"))
resize_layout.addWidget(self.spin_h)
resize_layout.addWidget(btn_resize)

# Информация об изображении
info_box = QGroupBox("Информация об изображении")
info_layout = QVBoxLayout(info_box)
self.lbl_info = QLabel("Файл не загружен")
```

```
        self.lbl_info.setWordWrap(True)
        info_layout.addWidget(self.lbl_info)

        left_layout.addWidget(btn_box)
        left_layout.addWidget(resize_box)
        left_layout.addWidget(info_box)
        left_layout.addStretch()

        self.image_view = ImageAreaWidget(self)

        splitter.addWidget(left_panel)
        splitter.addWidget(self.image_view)
        splitter.setSizes([300, 700])

    def save_state_for_undo(self):
        if self.current_pil_image:
            self.previous_pil_image = self.current_pil_image.copy()
            self.btn_undo.setEnabled(True)

    def update_image_display(self):
```

```
        if self.current_pil_image:
            self.image_view.set_image(self.current_pil_image)
            self.spin_w.setValue(self.current_pil_image.width)
            self.spin_h.setValue(self.current_pil_image.height)

            info_text = (
                f"<b>Файл:</b> {os.path.basename(self.current_file_path)}<br>"
                        f"<b>Разрешение:</b>  {self.current_pil_image.width}  x
{self.current_pil_image.height} px<br>"
                f"<b>Цветовой режим:</b> {self.current_pil_image.mode}"
            )
            self.lbl_info.setText(info_text)

    def load_image_from_path(self, path: str):
        try:
            info = ImageProcessor.get_info(path)
            self.current_pil_image = Image.open(path)
            self.current_file_path = path
            self.previous_pil_image = None
            self.btn_undo.setEnabled(False)
```

```
                self.update_image_display()


                self.settings["last_path"] = path
                AppStateManager.save_settings(self.settings)
                AppStateManager.append_history("Загрузка файла", path)
                logging.info(f"Успешно загружен файл: {path}")

            except Exception as e:
                QMessageBox.critical(self, "Ошибка", f"Не удалось открыть файл:\n{str(e)}")
                logging.error(f"Ошибка загрузки файла {path}: {str(e)}")

        def on_load_clicked(self):
            last_dir = os.path.dirname(self.settings.get("last_path", ""))
            path, _ = QFileDialog.getOpenFileName(
                self, "Выберите изображение", last_dir, "Images (*.png *.jpg *.jpeg *.bmp)"
            )
            if path:
```

```
            self.load_image_from_path(path)


        def on_crop_clicked(self):
            if not self.current_pil_image:
                return
            box = self.image_view.crop_rect_pixels
            if not box:
                QMessageBox.warning(self, "Предупреждение", "Выделите прямоугольную область мышью на изображении.")
                return

            self.save_state_for_undo()
            self.current_pil_image = ImageProcessor.crop(self.current_pil_image, box)
            self.update_image_display()
            AppStateManager.append_history("Обрезка (Crop)", f"Область: {box}")

        def on_grayscale_clicked(self):
            if not self.current_pil_image:
                return
```

```
            self.save_state_for_undo()
                                                    self.current_pil_image =
ImageProcessor.convert_to_grayscale(self.current_pil_image)
            self.update_image_display()
                AppStateManager.append_history("Фильтр ч/б", "Конвертация в
Grayscale")


        def on_resize_clicked(self):
            if not self.current_pil_image:
                return
            w, h = self.spin_w.value(), self.spin_h.value()
            self.save_state_for_undo()
            self.current_pil_image = ImageProcessor.resize(self.current_pil_image, w,
h)
            self.update_image_display()
              AppStateManager.append_history("Изменение разрешения", f"Новый
размер: {w}x{h}")


        def on_undo_clicked(self):
            if self.previous_pil_image:
                self.current_pil_image = self.previous_pil_image.copy()
```

```
                self.previous_pil_image = None
                self.btn_undo.setEnabled(False)
                self.update_image_display()
                    AppStateManager.append_history("Отмена (Undo)", "Возврат к предыдущему состоянию")

            def on_save_clicked(self):
                if not self.current_pil_image:
                    return
                base_dir = os.path.dirname(os.path.abspath(__file__))
                default_out = os.path.join(base_dir, "output", "result.png")
                save_path, selected_filter = QFileDialog.getSaveFileName(
                        self, "Сохранить файл", default_out, "PNG (*.png);;JPEG (*.jpg *.jpeg)"
                )
                if save_path:
                    try:
                        fmt = "PNG" if ".png" in save_path.lower() else "JPEG"
                        self.current_pil_image.save(save_path, fmt)
```

```
                QMessageBox.information(self, "Успех", f"Файл успешно сохранен:\n{save_path}")
                AppStateManager.append_history("Сохранение", save_path)
            except Exception as e:
                QMessageBox.critical(self, "Ошибка", f"Ошибка при сохранении файла:\n{str(e)}")

    def closeEvent(self, event):
        self.settings["window_size"] = [self.width(), self.height()]
        AppStateManager.save_settings(self.settings)
        super().closeEvent(event)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec_())
```

ФАЙЛ.2 test_image_processor.py:

```
import unittest
import os
from PIL import Image
from project_3_app import ImageProcessor


class TestImageProcessor(unittest.TestCase):
    """Модульные тесты для класса ImageProcessor."""

    def setUp(self):
        """Подготовка тестового изображения."""
                                                    self.test_filename =
os.path.join(os.path.dirname(os.path.abspath(__file__)), "test_temp_img.png")
        self.img_width = 200
        self.img_height = 100
            self.img = Image.new("RGB", (self.img_width, self.img_height),
color="blue")
        self.img.save(self.test_filename)

    def tearDown(self):
```

```
        """Очистка за собой."""
        if os.path.exists(self.test_filename):
            os.remove(self.test_filename)

    def test_01_get_info_success(self):
        """1. Проверка правильности чтения метаданных изображения."""
        info = ImageProcessor.get_info(self.test_filename)
        self.assertEqual(info["width"], self.img_width)
        self.assertEqual(info["height"], self.img_height)
        self.assertEqual(info["format"], "PNG")

    def test_02_get_info_file_not_found(self):
        """2. Проверка генерации исключения для несуществующего файла."""
        with self.assertRaises(FileNotFoundError):
            ImageProcessor.get_info("non_existing_file_999.png")

    def test_03_convert_to_grayscale(self):
        """3. Проверка преобразования цвета в оттенки серого."""
        gray_img = ImageProcessor.convert_to_grayscale(self.img)
```

```
        self.assertIsNotNone(gray_img)
        self.assertEqual(gray_img.size, (self.img_width, self.img_height))

    def test_04_crop_image(self):
        """4. Проверка кадрирования (обрезки)."""
        crop_box = (10, 10, 60, 50)
        cropped_img = ImageProcessor.crop(self.img, crop_box)
        self.assertEqual(cropped_img.width, 50)
        self.assertEqual(cropped_img.height, 40)

    def test_05_resize_image(self):
        """5. Проверка масштабирования."""
        new_w, new_h = 400, 300
        resized_img = ImageProcessor.resize(self.img, new_w, new_h)
        self.assertEqual(resized_img.width, new_w)
        self.assertEqual(resized_img.height, new_h)

if __name__ == "__main__":
    unittest.main()
```